UNIVERSITÉ – MASTER 2 INFORMATIQUE

Option : Systèmes et Réseaux

Prédiction de la disponibilité des ressources dans un système de calcul sur machines volontaires en environnements à ressources limitées

---

Document de conception et plan de collecte de données

Encadreur Dr ADAMOU HAMZA

Étudiant [Prénom NOM]

Année 2024 – 2025

Document de travail – Version 1.0

Table des matières

1 Présentation du sujet **4**

1.1 Contexte général 4

1.2 Question de recherche 4

1.3 Objectifs 5

1.3.1 Objectif général 5

1.3.2 Objectifs spécifiques 5

1.4 Résultats attendus 5

2 Lacunes identifiées et questions de recherche **6**

2.1 Les quatre lacunes de la littérature 6

2.1.1 Lacune 1 – Absence de traces africaines 6

2.1.2 Lacune 2 – Absence de mesure d'empreinte du modèle prédictif 6

2.1.3 Lacune 3 – Absence de modélisation de l'instabilité électrique . 7

2.1.4 Lacune 4 – Absence d'approche TinyML appliquée au VC . . . 7

2.2 Questions de recherche dérivées 7

3 Architecture du système de collecte **8**

3.1 Vue d'ensemble du système 8

3.2 L'agent local 10

3.2.1 Principes de fonctionnement 10

3.2.2 Mécanisme de détection des coupures de courant 10

3.2.3 Tâches synthétiques contrôlées 11

3.2.4 Compatibilité multi-OS 11

4 Format des données collectées **12**

4.1 Vue d'ensemble du schéma relationnel 12

4.2 Table `machines` – Profil de la machine 12

4.3 Table `sessions` – Cycle de vie d'une session 13

4.4 Table `snapshots` – Mesures périodiques (table centrale) 13

4.4.1 Volume de données attendu 15

4.5 Table `power_events` – Événements de coupure électrique 15

4.6 Table `sync_log` – Traçabilité des synchronisations 16

5 Architecture du modèle de prédiction **17**

5.1 Architecture à deux niveaux GM/LM 17

5.2 Features d'entrée du modèle GRU 18

5.3 Contraintes frugalité (TinyML) 19

6 Portail web – Dashboard et espace volontaires **20**

6.1 Vue d'ensemble du portail 20

6.2 Espace administration 20

6.2.1 Tableau de bord temps réel 20

6.2.2 Export du dataset 21

6.3 Espace volontaires – Téléchargement guidé 21

6.3.1 Principe de conception 21

6.3.2 Guide d'installation Windows – Étapes illustrées 22

6.3.3 Guide d'installation Linux – Étapes illustrées 23

7 Plan de collecte de données **24**

7.1 Stratégie de collecte 24

7.2 Plan de déploiement 24

7.3 Critères de qualité du dataset 25

7.4 Éthique et anonymisation 25

8 Synthèse et prochaines étapes **26**

8.1 Récapitulatif des contributions originales 26
8.2 Séquencement des prochaines étapes 26

Chapitre 1

Présentation du sujet

1.1 Contexte général

Dans les environnements à ressources limitées, notamment dans les pays en développement, l'accès aux supercalculateurs demeure rare et coûteux. Le **calcul sur machines volontaires** (*Volunteer Computing*, VC) représente alors une alternative stratégique : il exploite des ordinateurs personnels, serveurs institutionnels et smartphones sous-utilisés pour constituer une puissance de calcul distribuée, sur le modèle des plateformes BOINC et SETI@home.

Ce modèle fait néanmoins face à une difficulté majeure : la **disponibilité imprévisible** des machines volontaires. Les interruptions sont fréquentes et causées par :

— l'instabilité électrique (délestages fréquents dans les pays en développement) ;
— l'usage concurrent par les propriétaires des machines ;
— la variabilité des connexions réseau.

Cette volatilité rend difficile la planification des tâches distribuées et réduit l'efficacité globale du système.

1.2 Question de recherche

Question de recherche centrale

Comment concevoir un modèle prédictif frugal et efficace, capable d'anticiper la disponibilité des machines volontaires dans un environnement à ressources limitées ?

1.3 Objectifs

1.3.1 Objectif général

Concevoir, implémenter et évaluer un modèle prédictif léger et optimisé, permettant d'améliorer la fiabilité et l'efficacité des systèmes de calcul sur machines volontaires.

1.3.2 Objectifs spécifiques

1. Analyser les caractéristiques des ressources volontaires (CPU, mémoire, énergie, connectivité) et collecter des données réelles sur leur disponibilité.
2. Concevoir un modèle prédictif frugal minimisant l'utilisation CPU et mémoire.
3. Implémenter et tester ce modèle dans un réseau représentatif de machines volontaires.
4. Évaluer les performances en termes de précision de prédiction, robustesse face à la volatilité et coût en ressources.
5. Comparer les compromis entre précision, consommation et rapidité par rapport à d'autres approches existantes.

1.4 Résultats attendus

Livrables du projet

— Un modèle prédictif opérationnel anticipant la disponibilité des ressources avec une précision élevée (cible : F1 $\geq$ 0,80).

— Une empreinte mémoire et CPU minimale : RAM $\leq$ 10 Mo, inférence $\leq$ 100 ms, CPU $\leq$ 5%.

— Un prototype fonctionnel déployable dans un vrai système de calcul volontaire.

— Des recommandations pratiques pour intégrer ce modèle dans des plateformes de calcul distribué en contexte africain.

Chapitre 2

Lacunes identifiées et questions de recherche

L'analyse critique de la littérature (articles 2019–2026, orientation IA/ML) permet d'identifier quatre lacunes majeures qui constituent le coeur de l'originalité de ce travail.

2.1 Les quatre lacunes de la littérature

2.1.1 Lacune 1 – Absence de traces africaines

Aucune publication existante n'utilise des traces de disponibilité collectées dans des institutions africaines. Les patterns locaux (délestages électriques, usage concurrent en salle informatique, calendrier universitaire africain) sont **structurellement différents** des traces SETI@home collectées dans des contextes nord-américains ou européens.

2.1.2 Lacune 2 – Absence de mesure d'empreinte du modèle prédictif

Aucune étude ne mesure simultanément la précision de prédiction et l'empreinte mémoire/CPU du modèle lui-même, rendant impossible l'évaluation de son adéquation à des environnements contraints. Les métriques habituellement reportées (Accuracy, F1, AUC) ignorent totalement le coût computationnel de l'inférence.

2.1.3 Lacune 3 – Absence de modélisation de l'instabilité électrique

Les délestages électriques ne sont *jamais* modélisés comme variable d'entrée ou perturbation dans les modèles de prédiction de disponibilité VC. Ils sont traités comme du bruit, alors qu'en contexte africain ils constituent un signal structurant prévisible (calendrier de délestage, plages horaires récurrentes).

2.1.4 Lacune 4 – Absence d'approche TinyML appliquée au VC

Bien que le TinyML soit un domaine en pleine expansion, aucune publication n'applique ses techniques (quantification, pruning, distillation) à la prédiction de disponibilité dans les systèmes VC.

2.2 Questions de recherche dérivées

Ces lacunes se traduisent en trois questions de recherche opérationnelles :

QR1 – Performance du modèle GRU compact

Un modèle GRU compact (1 couche, quantifié INT8) peut-il atteindre F1 $\geq$ 0,80 sur des traces africaines de disponibilité VC avec une empreinte RAM $<$ 10 Mo ?

QR2 – Impact des délestages sur les distributions

Dans quelle mesure les délestages électriques modifient-ils les distributions de disponibilité par rapport aux traces SETI@home, et comment un modèle peut-il s'y adapter ?

QR3 – Compromis précision / empreinte

Quel compromis précision / empreinte computationnelle est acceptable pour l'ordonnancement de tâches dans un système VC en contexte africain ?

Chapitre 3

Architecture du système de collecte

3.1 Vue d'ensemble du système

Le système de collecte est composé de trois couches distinctes : un **agent local** déployé sur chaque machine volontaire, un **serveur central** avec base de données et API REST, et un **portail web** combinant dashboard d'administration et interface de téléchargement pour les volontaires.

Architecture globale du système de collecte

```
MACHINES VOLONTAIRES (N machines)

Agent local (Python)

Collecte     Heartbeat     Tâches
/5 min       fichier       synthétiques

SQLite local (buffer offline)

HTTP POST batch (si connecté)

SERVEUR CENTRAL

API REST                PostgreSQL
(Flask/FastAPI)    (machines, sessions,
                        snapshots, power_events,
     sync_log)

PORTAIL WEB

Dashboard admin          Espace volontaires
- Visualisation          - Choix OS
- Statistiques           - Téléchargement agent
- Export ZIP/CSV         - Guide installation
```

FIGURE 3.1 – Architecture globale du système de collecte de données

3.2 L'agent local

3.2.1 Principes de fonctionnement

L'agent local est un programme Python léger fonctionnant en arrière-plan, invisible pour l'utilisateur. Il respecte les contraintes suivantes :

— **Frugalité** : consommation CPU $< 1\%$ en mode veille, RAM < 30 Mo.
— **Résilience** : stockage local SQLite pour ne perdre aucune donnée en cas de coupure réseau.
— **Transparence** : aucune interférence avec l'usage normal de la machine par son propriétaire.
— **Détection des coupures** : mécanisme heartbeat pour identifier les arrêts brutaux (délestages électriques).

3.2.2 Mécanisme de détection des coupures de courant

La détection des coupures est réalisée via un fichier heartbeat local, mis à jour à chaque collecte. Au redémarrage, l'agent compare l'heure du dernier heartbeat avec l'heure de démarrage actuelle :

```python
import json, time, psutil
from pathlib import Path

HEARTBEAT_FILE = Path("heartbeat.json")

def write_heartbeat():
    """Ecrire le heartbeat a chaque collecte (toutes les 5 min)."""
    data = {
        "last_ts": time.time(),
        "shutdown_clean": False  # mis a True avant arret propre
    }
    HEARTBEAT_FILE.write_text(json.dumps(data))

def detect_power_cut():
    """Verifier au demarrage si la derniere session s'est arretee
       brutalement."""
    if not HEARTBEAT_FILE.exists():
        return None  # premier demarrage

    data = json.loads(HEARTBEAT_FILE.read_text())
    boot_time = psutil.boot_time()
    last_hb   = data["last_ts"]
```

```
    gap_s      = boot_time - last_hb

    if data["shutdown_clean"]:
        return {"type": "clean", "gap_s": 0}
    elif gap_s > 60:           # seuil : 1 minute
        return {"type": "power_cut", "gap_s": int(gap_s)}
    else:
        return {"type": "unknown", "gap_s": int(gap_s)}
```

Listing 3.1 – Logique de détection de coupure de courant

3.2.3 Tâches synthétiques contrôlées

Lorsque la machine est sous-utilisée (inactive depuis plus de 5 minutes, CPU $< 30\%$), l'agent peut lancer des tâches de calcul fictives pour simuler la charge d'un vrai client VC. Le garde-fou garantit que ces tâches ne nuisent jamais à l'usage normal :

```
import psutil

SEUIL_CPU  = 30    # % CPU maximum autorise
SEUIL_RAM  = 40    # % RAM maximum autorisee
IDLE_MIN_S = 300   # inactivite minimale requise (5 min)

def safe_to_run_task(idle_seconds: int) -> bool:
    cpu_ok  = psutil.cpu_percent(interval=1) < SEUIL_CPU
    ram_ok  = psutil.virtual_memory().percent < SEUIL_RAM
    idle_ok = idle_seconds > IDLE_MIN_S
    return cpu_ok and ram_ok and idle_ok
```

Listing 3.2 – Garde-fou avant lancement d'une tâche synthétique

3.2.4 Compatibilité multi-OS

L'agent est distribué sous deux formes :

Système	Format	Mécanisme de démarrage auto
Windows	`.exe` (PyInstaller)	Clé de registre `HKCU\Run`
Linux	Binaire PyInstaller ou `.deb`	Service `systemd`

Chapitre 4

Format des données collectées

4.1 Vue d'ensemble du schéma relationnel

La base de données est composée de cinq tables. La même structure est utilisée localement (SQLite sur chaque machine) et sur le serveur central (PostgreSQL).

Principe de synchronisation

L'agent collecte et stocke localement dans SQLite. Dès qu'une connexion Internet est disponible, les lignes non synchronisées (`synced = False`) sont envoyées en lot (*batch*) au serveur central via l'API REST. Aucune donnée n'est perdue en cas de coupure réseau.

4.2 Table `machines` – Profil de la machine

Créée une seule fois au premier lancement de l'agent. L'identifiant est un hash SHA-256 de l'adresse MAC, garantissant l'anonymat tout en étant reproductible.

```sql
CREATE TABLE machines (
    machine_id      TEXT PRIMARY KEY,   -- SHA-256(MAC address)
    os              TEXT NOT NULL,      -- 'windows' | 'linux'
    hostname_hash   TEXT NOT NULL,      -- SHA-256(hostname), anonymise
    cpu_cores       INTEGER,
    ram_total_mb    INTEGER,
    timezone        TEXT,               -- ex: 'Africa/Douala'
    city            TEXT,               -- ex: 'Yaounde' (saisie volontaire)
    registered_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Listing 4.1 – DDL – Table machines

4.3 Table sessions – Cycle de vie d'une session

Une ligne est insérée à chaque démarrage de la machine. C'est dans cette table que réside l'information sur le type d'arrêt, répondant directement à la **lacune 3** identifiée par le directeur de mémoire.

```sql
CREATE TABLE sessions (
    session_id          TEXT PRIMARY KEY,    -- UUID v4
    machine_id          TEXT NOT NULL REFERENCES machines(machine_id),
    boot_time           DATETIME NOT NULL,
    shutdown_time       DATETIME,            -- NULL si session encore active
    shutdown_type       TEXT,                -- 'clean' | 'power_cut' | 'unknown'
    power_cut_detected  BOOLEAN DEFAULT FALSE,
    power_cut_gap_s     INTEGER,             -- duree estimee de la coupure (s)
    uptime_seconds      INTEGER              -- duree totale de la session
);
```

Listing 4.2 – DDL – Table sessions

Valeurs possibles de `shutdown_type`

- `clean` : arrêt normal via le système d'exploitation.
- `power_cut` : gap entre dernier heartbeat et redémarrage supérieur à 60 secondes et `shutdown_clean = False`.
- `unknown` : gap court (< 60 s) sans arrêt propre détecté (hibernation, crash OS).

4.4 Table snapshots – Mesures périodiques (table centrale)

C'est la table principale, collectée toutes les **5 minutes**. Elle constitue le dataset d'entraînement du modèle GRU. Les colonnes `day_of_week` et `hour_of_day` sont calculées à la collecte et stockées directement ; elles servent de features d'entrée au modèle sans transformation supplémentaire.

```sql
CREATE TABLE snapshots (
    snapshot_id          INTEGER PRIMARY KEY AUTOINCREMENT,
```

```sql
    session_id            TEXT NOT NULL REFERENCES sessions(session_id),
    machine_id            TEXT NOT NULL REFERENCES machines(machine_id),

    -- Horodatage (double : UTC pour la BD, local pour l'analyse)
    ts_utc                DATETIME NOT NULL,
    ts_local              DATETIME NOT NULL,
    day_of_week           INTEGER NOT NULL,  -- 0=Lundi ... 6=Dimanche
    hour_of_day           INTEGER NOT NULL,  -- 0 .. 23

    -- Ressources CPU
    cpu_percent           REAL,
    cpu_freq_mhz          REAL,

    -- Ressources memoire
    ram_available_mb      INTEGER,
    ram_percent_used      REAL,

    -- Ressources disque
    disk_read_mbps        REAL,
    disk_write_mbps       REAL,

    -- Reseau et connectivite
    is_connected          BOOLEAN NOT NULL DEFAULT FALSE,
    network_latency_ms    REAL,              -- ping vers 8.8.8.8
    bytes_sent_kb         INTEGER,
    bytes_recv_kb         INTEGER,

    -- Activite utilisateur
    user_active           BOOLEAN,           -- saisie clavier/souris < 5 min
    idle_seconds          INTEGER,           -- secondes d'inactivite consecutive
    screen_on             BOOLEAN,

    -- Etat de l'agent
    synthetic_task_active BOOLEAN DEFAULT FALSE,
    synced                BOOLEAN DEFAULT FALSE,  -- envoye au serveur ?

    -- Index pour requetes frequentes
    CONSTRAINT chk_day CHECK (day_of_week BETWEEN 0 AND 6),
    CONSTRAINT chk_hour CHECK (hour_of_day BETWEEN 0 AND 23)
);

CREATE INDEX idx_snap_machine ON snapshots(machine_id, ts_utc);
```

```sql
CREATE INDEX idx_snap_synced  ON snapshots(synced) WHERE synced =
    FALSE;
```

Listing 4.3 – DDL – Table snapshots

4.4.1 Volume de données attendu

Paramètre	Valeur	Calcul
Fréquence de collecte	5 min	288 snapshots/jour/machine
Nombre de machines visées	30	–
Durée de collecte	8 semaines	56 jours
Total snapshots	**≈ 484 000**	$30 \times 288 \times 56$
Taille estimée (SQLite)	**≈ 80 Mo**	–
Taille export CSV compressé	**≈ 8 Mo**	compression ×10

4.5 Table `power_events` – Événements de coupure électrique

Table dédiée aux coupures de courant. Elle répond directement à la **lacune 3** du directeur en traitant les délestages comme des événements de première classe, non comme du bruit.

```sql
CREATE TABLE power_events (
    event_id            INTEGER PRIMARY KEY AUTOINCREMENT,
    machine_id          TEXT NOT NULL REFERENCES machines(machine_id),
    detected_at         DATETIME NOT NULL,    -- heure du redemarrage
    last_heartbeat_ts   DATETIME NOT NULL,    -- dernier heartbeat connu
        avant coupure
    gap_seconds         INTEGER NOT NULL,     -- duree estimee de la
        coupure
    cpu_before          REAL,                -- charge CPU au dernier
        heartbeat
    ram_before          REAL                 -- % RAM au dernier
        heartbeat
);
```

Listing 4.4 – DDL – Table power_events

Utilisation pour QR2

La comparaison des distributions de disponibilité avec vs. sans événements `power_cut` permettra de répondre à la question de recherche 2 : dans quelle mesure les délestages modifient-ils les patterns de disponibilité ?

4.6 Table `sync_log` – Traçabilité des synchronisations

```sql
CREATE TABLE sync_log (
    sync_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    machine_id   TEXT NOT NULL REFERENCES machines(machine_id),
    synced_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    rows_sent    INTEGER NOT NULL,
    status       TEXT NOT NULL,  -- 'ok' | 'fail' | 'partial'
    error_msg    TEXT            -- NULL si status='ok'
);
```

Listing 4.5 – DDL – Table sync_log

Chapitre 5

Architecture du modèle de prédiction

5.1 Architecture à deux niveaux GM/LM

Le modèle de prédiction suit une architecture hiérarchique à deux niveaux, répondant simultanément aux contraintes de frugalité et à la gestion du problème de démarrage à froid (*cold start*) :

Architecture GM/LM – Modèle à deux niveaux

```
                MODÈLE GLOBAL (GM)
Entraîné sur TOUTES les machines


  k-means              GRU compact
  clustering     1 couche, hidden=64
  (profils             Quantifié INT8 (TFLite)
  de machines)         RAM cible : < 10 Mo


           Cold start : pas d'historique local
           Après seuil S* snapshots locaux


              MODÈLE LOCAL (LM_i)
Fine-tuning par machine, online learning

Activé après S* snapshots (à déterminer
expérimentalement, hypothèse : S* = 288 = 1 jour)

Adapté au comportement propre de la machine i
```

FIGURE 5.1 – Architecture à deux niveaux GM/LM

5.2 Features d'entrée du modèle GRU

Les features d'entrée sont directement disponibles dans la table `snapshots` sans transformation majeure :

Feature	Source	Justification
`hour_of_day`	snapshots	Patterns d'usage horaires
`day_of_week`	snapshots	Patterns hebdomadaires (week-end)
`cpu_percent`	snapshots	Charge courante
`ram_percent_used`	snapshots	Charge mémoire
`idle_seconds`	snapshots	Probabilité de libération imminente
`user_active`	snapshots	Occupation par le propriétaire
`is_connected`	snapshots	Disponibilité réseau
`power_cut_flag`	sessions	Contexte de délestage (**lacune 3**)

5.3 Contraintes frugalité (TinyML)

Métrique	Cible	Technique
RAM du modèle	< 10 Mo	Quantification INT8 (TFLite)
Temps d'inférence	< 100 ms	1 seule couche GRU
CPU inférence	$< 5\%$	Priorité basse du thread
Précision (F1)	$\geq 0{,}80$	Fine-tuning local

La quantification INT8 réduit la taille du modèle d'un facteur 4 (float32 $\rightarrow$ int8) avec une perte de précision typiquement inférieure à 2 points de F1.

Chapitre 6

Portail web – Dashboard et espace volontaires

6.1 Vue d'ensemble du portail

Le portail web remplit deux fonctions distinctes, accessibles depuis la même interface :

1. **Espace administration (privé)** : visualisation temps réel, statistiques, export du dataset.
2. **Espace volontaires (public)** : téléchargement guidé de l'agent, avec assistant d'installation adapté au système d'exploitation.

6.2 Espace administration

6.2.1 Tableau de bord temps réel

Le dashboard affiche :

— Nombre de machines actives en temps réel.
— Graphiques de disponibilité par machine et par heure.
— Carte de chaleur (*heatmap*) des coupures de courant (jour × heure).
— Statistiques de synchronisation (machines connectées, dernière sync).
— Alertes sur les machines silencieuses depuis plus de 24 heures.

6.2.2 Export du dataset

Le dashboard permet d'exporter le dataset complet sous forme d'un fichier ZIP contenant :

Fichier dans le ZIP	Contenu
`machines.csv`	Profils anonymisés de toutes les machines
`sessions.csv`	Toutes les sessions avec type d'arrêt
`snapshots.csv`	Dataset principal (features + labels)
`power_events.csv`	Tous les événements de coupure détectés
`README.txt`	Description du format et des colonnes

6.3 Espace volontaires – Téléchargement guidé

6.3.1 Principe de conception

L'espace volontaires est conçu selon le principe : **zéro souffrance d'installation**. Le volontaire doit pouvoir installer et lancer l'agent en moins de 3 minutes, sans connaissances techniques particulières.

Le flux de l'espace volontaires est le suivant :

Flux de l'espace volontaires

```
[Page d'accueil volontaires]



[Étape 1] Choisir votre système d'exploitation

          Windows           Linux




[Étape 2] Bouton de téléchargement (gros, visible)

     Télécharger l'agent (Windows .exe)
     ou
     Télécharger l'agent (Linux .deb/.bin)



[Étape 3] Guide d'installation illustré (images)
          → adapté automatiquement au SE choisi


[Confirmation] L'agent se connecte au serveur
               → le volontaire apparaît dans le dashboard
```

FIGURE 6.1 – Flux de l'espace volontaires

6.3.2 Guide d'installation Windows – Étapes illustrées

Le guide Windows est présenté sous forme d'un assistant pas-à-pas avec captures d'écran simulées :

1. **Télécharger** : clic sur le bouton de téléchargement → fichier `vc-agent-setup.exe` sauvegardé dans `Téléchargements`.
2. **Lancer** : double-clic sur le fichier. Si Windows affiche une alerte de sécurité, cliquer sur *« Informations complémentaires »* puis *« Exécuter quand même »*.
3. **Installer** : cliquer sur *« Installer »* et attendre 30 secondes.
4. **Terminé** : une icône apparaît dans la barre système. L'agent tourne en arrière-plan et démarre automatiquement avec Windows.

6.3.3 Guide d'installation Linux – Étapes illustrées

1. **Télécharger** : clic sur le bouton → fichier `vc-agent.deb` téléchargé.
2. **Installer (méthode graphique)** : double-clic sur le fichier `.deb` → le gestionnaire de paquets s'ouvre → cliquer sur *« Installer »*.
3. **Installer (méthode terminal)** : ouvrir un terminal et saisir :

```
sudo dpkg -i vc-agent.deb
sudo systemctl enable --now vc-agent
```

4. **Vérifier** : l'agent est actif. On peut le vérifier avec :

```
systemctl status vc-agent
```

Principe de non-nuisance garanti

L'agent n'utilise jamais plus de 1% de CPU en mode collecte normale. Les tâches synthétiques s'arrêtent dès que l'utilisateur reprend sa machine (activité clavier ou souris détectée). Le programme peut être désinstallé à tout moment en un clic.

Chapitre 7

Plan de collecte de données

7.1 Stratégie de collecte

La stratégie de collecte combine deux sources complémentaires :

Source	Avantage	Rôle dans le mémoire
Traces publiques (FTA, BOINC, Google Cluster Traces)	Disponibles immédiatement, validées par la communauté scientifique	Référence comparative, entraînement initial du GM
Collecte terrain locale (machines de l'institution)	Données africaines originales, délestages réels modélisés	Contribution originale, réponse à la lacune 1

7.2 Plan de déploiement

Phase	Durée	Actions
Développement agent	S1 – S3	Code Python, packaging, tests
Déploiement pilote	S3 – S4	5 machines test, validation du schéma
Collecte principale	S4 – S10	30+ machines, 6 semaines
Analyse exploratoire	S8 – S12	Statistiques, réponse QR2
Modélisation	S10 – S16	Entraînement GRU, quantification
Rédaction finale	S12 – S20	Chapitres mémoire

7.3 Critères de qualité du dataset

Pour que le dataset soit exploitable scientifiquement, les critères suivants doivent être atteints :

- **Couverture temporelle** : au moins 4 semaines continues par machine, incluant des week-ends et des jours fériés.
- **Couverture des événements** : au moins 20 événements `power_cut` documentés dans la table `power_events`.
- **Complétude** : taux de snapshots manquants $< 5\%$ (hors coupures électriques).
- **Diversité** : machines hétérogènes (RAM de 2 à 16 Go, OS Windows et Linux, usages variés).

7.4 Éthique et anonymisation

Principes éthiques de la collecte

- **Consentement explicite** : chaque volontaire accepte une charte avant de télécharger l'agent.
- **Anonymisation** : aucun nom, aucune adresse IP, aucun contenu utilisateur n'est collecté. Seuls des hashes non-réversibles sont utilisés comme identifiants.
- **Droit de retrait** : le volontaire peut désinstaller l'agent et demander la suppression de ses données à tout moment.
- **Transparence** : le code source de l'agent est disponible pour consultation.

Chapitre 8

Synthèse et prochaines étapes

8.1 Récapitulatif des contributions originales

Ce travail se distingue de la littérature existante sur quatre points :

1. **Traces africaines originales** : première collecte de disponibilité VC dans une institution universitaire africaine, intégrant les patterns locaux (délestages, calendrier universitaire).
2. **Métrique composite précision/empreinte** : évaluation simultanée du F1-score et de l'empreinte mémoire/CPU du modèle, créant un nouveau référentiel d'évaluation adapté aux environnements contraints.
3. **Modélisation explicite des délestages** : les coupures électriques sont traitées comme variable d'entrée du modèle, via la colonne `power_cut_flag` dérivée de la table `power_events`.
4. **Approche TinyML appliquée au VC** : première application de la quantification INT8 (TensorFlow Lite) à la prédiction de disponibilité dans les systèmes VC.

8.2 Séquencement des prochaines étapes

Étape	**Priorité**	**Livrable**
1. Finalisation état de l'art	Immédiate	Chapitre 2 du mémoire
2. Développement agent Python	Immédiate	Agent fonctionnel Win+Linux
3. Déploiement portail web	S3	Dashboard + espace volontaires
4. Collecte terrain	S4	Dataset $\geq$ 30 machines
5. Analyse exploratoire	S8	Réponse à QR2
6. Entraînement GRU + quantification	S10	Modèle INT8, métriques QR1
7. Courbe de Pareto précision/empreinte	S14	Réponse à QR3
8. Rédaction finale	S16	Mémoire complet